浏览器底层原理：从第一性原理理解 URL 到页面渲染

视频定位

这期视频面向有一定计算机基础、准备前端或后端面试、希望真正理解浏览器底层机制的观众。核心目标不是背诵“输入 URL 后发生了什么”，而是用第一性原理回答一个更本质的问题：

浏览器如何把远程服务器返回的一堆字节，变成一个安全、可交互、能高性能更新的页面？

开场

很多人理解浏览器，会把它简化成一句话：输入一个网址，然后打开一个网页。

但如果我们从底层看，浏览器远不只是“打开网页的软件”。它更像一个小型操作系统：它要访问网络，要管理进程和线程，要解析多种语言，要执行 JavaScript，要调用 GPU，要处理用户输入，还要防止一个恶意网页读取你的本地文件、窃取其他网站的登录态，或者拖垮整台机器。

所以这期视频我们不按碎片化知识点讲，而是从第一性原理出发，重新理解浏览器。

浏览器要解决的根问题只有一个：

如何把不可信的远程资源，安全、高效、正确地运行在用户的本机上？

这句话里面有三个关键词。

第一，远程资源。网页不是本地程序，它大多数时候来自服务器，所以浏览器首先是一个网络客户端。

第二，不可信。你访问的网站不一定可靠，网页里的脚本也不一定善良，所以浏览器必须有安全边界。

第三，运行。网页不是一张静态图片，而是一个会执行代码、响应交互、动态更新的应用。

围绕这三个关键词，浏览器内部就自然分裂出六个核心任务：找资源、取资源、理解资源、渲染页面、执行程序、保护用户。

第一部分：URL 的本质是资源定位问题

用户输入一个 URL，比如：

```
https://www.example.com/article?id=1
```

从第一性原理看，URL 不是“网址”这么简单，它是一个资源定位符。它告诉浏览器四件事：

协议是什么：这里是 HTTPS，意味着要使用 HTTP 语义，并且传输层要通过 TLS 加密。

主机是谁：这里是 www.example.com，浏览器需要找到它背后的服务器地址。

路径是什么：这里是 /article，告诉服务器你要哪个资源。

查询参数是什么：这里是 id=1，告诉服务器这次请求的附加条件。

浏览器拿到 URL 后，第一步不是立刻发 HTTP 请求，而是先判断：这个资源在哪里？应该怎么连过去？有没有缓存可以直接用？

如果本地缓存命中，并且缓存仍然有效，浏览器甚至不需要访问网络。否则，它需要进入 DNS 查询。

DNS 的本质，是把人类可读的域名翻译成机器可连接的 IP 地址。浏览器可能会先查自己的 DNS 缓存，再查操作系统缓存，再通过配置的 DNS 服务器递归查询。

拿到 IP 地址后，浏览器还不能直接拿到网页。它必须先建立连接。

如果是 HTTP/1.1 或 HTTP/2 over TCP，浏览器需要完成 TCP 三次握手。三次握手的本质，是客户端和服务器确认双方都具备发送和接收能力，并协商初始序列号。

如果是 HTTPS，还要在 TCP 之上完成 TLS 握手。TLS 的本质不是“加密”两个字，而是解决三个问题：我连的是不是正确的服务器？传输内容会不会被窃听？内容会不会被篡改？

完成这些之后，浏览器才会发送 HTTP 请求。

HTTP 请求本质上是一段结构化文本，告诉服务器：我要什么资源，我能接受什么格式，我带了哪些 Cookie，我的缓存版本是什么。

服务器返回 HTTP 响应，里面包括状态码、响应头和响应体。状态码告诉浏览器请求结果，响应头描述资源类型、缓存策略、安全策略、压缩方式，响应体才是真正的 HTML、CSS、JavaScript、图片或其他资源。

所以，“输入 URL 后发生了什么”的前半段，本质上是：

把一个抽象资源名，经过缓存、DNS、连接、加密、请求响应，变成本机拿到的一串字节。

第二部分：浏览器为什么是多进程架构

拿到字节之后，浏览器要开始解释它。但在解释之前，我们先问一个更底层的问题：浏览器为什么不只是一个进程？

答案来自两个约束：稳定性和安全性。

网页是不可信的。如果所有网页都跑在同一个进程里，一个页面崩溃可能拖垮整个浏览器；一个恶意页面也可能更容易攻击其他页面。现代浏览器通常采用多进程架构，把不同职责隔离开。

典型浏览器里至少有几类进程。

Browser Process，负责浏览器主控逻辑，比如地址栏、标签页管理、权限管理、进程调度。

Renderer Process，负责具体网页的解析、布局、绘制和 JavaScript 执行。通常不同站点会尽量运行在不同渲染进程里，这就是站点隔离的基础。

Network Service，负责网络请求、代理、缓存、Cookie 等网络相关工作。

GPU Process，负责和显卡交互，处理合成、光栅化等图形任务。

从第一性原理看，多进程不是为了“架构高级”，而是为了建立故障边界和安全边界。

一个标签页卡死，不应该让整个浏览器卡死。一个恶意站点，也不应该轻易读取另一个站点的数据。浏览器通过进程隔离、沙箱、权限控制，把风险限制在尽可能小的范围内。

这也解释了为什么浏览器很吃内存。它不是一个单体程序，而是在为稳定性和安全性付出资源成本。

第三部分：HTML 不是页面，而是页面的描述

服务器返回 HTML 后，很多人会说“浏览器开始渲染 HTML”。这句话没错，但不够精确。

HTML 本身不是页面，它只是页面结构的文本描述。浏览器要先把这段文本解析成 DOM。

DOM，也就是 Document Object Model，本质上是一棵树。HTML 里的标签会变成一个个节点，节点之间形成父子关系。比如 body 下面有 div，div 下面有 p，p 里面有文本。

为什么要变成树？因为页面结构天然是嵌套的。树结构可以表达包含关系，也方便 JavaScript 后续查询、修改和删除节点。

浏览器解析 HTML 时，并不是等整个 HTML 下载完才开始处理，而是边下载边解析。遇到外部 CSS，会发起 CSS 请求；遇到图片，也会发起图片请求；遇到 script 标签，则可能暂停 HTML 解析。

为什么 JavaScript 会阻塞解析？因为 JavaScript 有能力修改 DOM。如果浏览器一边继续解析后面的 HTML，一边执行前面的脚本，而脚本又调用 document.write 或修改节点，页面结构就可能发生变化。为了保证语义正确，普通同步脚本默认会阻塞 HTML 解析。

这就是 async 和 defer 的意义。

async 表示脚本下载不阻塞 HTML 解析，但下载完成后会尽快执行，执行时仍可能打断解析。它适合彼此独立、和 DOM 顺序无关的脚本。

defer 表示脚本下载不阻塞 HTML 解析，并且等 DOM 解析完成后按顺序执行。它更适合依赖页面结构的业务脚本。

所以，HTML 解析的底层逻辑不是“读文件”，而是：

把字节流逐步转换成可操作的结构树，同时发现并调度更多资源。

第四部分：CSS 决定元素如何被计算

DOM 只描述页面有什么，但不描述它长什么样。样式来自 CSS。

浏览器会把 CSS 文本解析成 CSSOM，也就是 CSS Object Model。它同样是一种结构化表示，但它解决的是另一个问题：每个 DOM 节点最终应该应用哪些样式？

CSS 的复杂性在于层叠。一个元素的最终样式，可能来自浏览器默认样式、外部样式表、内联样式、继承属性、选择器优先级、媒体查询，甚至运行时动态修改。

所以 CSS 不是简单地“读一条规则，应用一条规则”。浏览器要计算每个节点的最终样式，这个过程叫 style calculation。

当 DOM 和 CSSOM 都准备好后，浏览器会构建 Render Tree。

Render Tree 不是 DOM 的完整复制。它只包含需要被渲染出来的节点。比如 display: none 的元素存在于 DOM 中，但不会进入 Render Tree，因为它不参与布局和绘制。

这里要区分两个概念。

DOM 关注语义结构：页面有哪些节点。

Render Tree 关注视觉结构：哪些节点需要被画出来，以及它们的最终样式是什么。

这一步之后，浏览器终于知道“要画哪些东西”，但还不知道“画在哪里”。

第五部分：Layout、Paint、Composite 是渲染管线的核心

渲染页面不是一步完成的。它至少包含三个关键阶段：Layout、Paint、Composite。

Layout，也叫 Reflow，解决的是几何问题：每个元素的位置和尺寸是多少？

一个 div 的宽高，不一定写死在 CSS 里。它可能依赖父容器宽度、内容大小、字体、图片尺寸、视口大小、盒模型、flex 或 grid 规则。浏览器必须根据布局算法计算出每个盒子的几何信息。

Paint 解决的是像素绘制问题：每个元素应该以什么顺序、什么颜色、什么边框、什么阴影画出来？

浏览器不会直接把所有内容一次性画到屏幕上。它会生成绘制指令，描述背景、文字、边框、图片等如何绘制。

Composite 解决的是图层合成问题：哪些内容可以放到独立图层，最后由合成器组合成最终画面？

合成的意义在于性能。如果一个元素只是 transform 或 opacity 变化，浏览器有机会跳过 Layout 和 Paint，只在合成阶段移动或透明化已有图层。这就是为什么动画性能优化经常建议优先使用 transform 和 opacity，而不是频繁修改 width、height、top、left。

现在我们可以理解三类性能成本。

修改 width、height、font-size 这类影响几何的属性，通常会触发 Layout，成本最高。

修改 color、background、box-shadow 这类不影响几何但影响外观的属性，通常会触发 Paint。

修改 transform、opacity 这类可合成属性，理想情况下只触发 Composite，成本最低。

这也是所谓 layout thrashing 的来源。如果 JavaScript 在一个循环里反复读取布局信息，再修改样式，再读取，再修改，浏览器就可能被迫多次同步计算布局，性能会急剧下降。

渲染管线的本质是：

从结构和样式出发，计算几何，生成绘制指令，再通过合成把结果送到屏幕。

第六部分：JavaScript 让页面从文档变成应用

如果只有 HTML 和 CSS，网页更像一份可以排版的文档。JavaScript 的出现，让网页变成了应用。

浏览器中的 JavaScript 通常由 V8、SpiderMonkey、JavaScriptCore 这类引擎执行。以 V8 为例，它负责解析 JavaScript 源码，生成抽象语法树，编译成字节码或机器码，并在运行时管理对象、函数调用和垃圾回收。

JavaScript 运行时有几个核心概念。

调用栈用来管理函数调用。一个函数调用另一个函数，新的调用帧会压入栈；函数返回后，对应调用帧弹出。

堆用来存放对象、数组、闭包等动态分配的数据。垃圾回收器会在合适时机回收不再可达的对象。

但浏览器里的 JavaScript 不是孤立运行的。它还可以调用 DOM API、定时器、网络请求、事件监听等能力。这些能力很多不是 JavaScript 引擎本身提供的，而是浏览器环境提供的 Web APIs。

这就引出事件循环。

JavaScript 主线程一次只能执行一个任务。如果遇到 setTimeout、fetch、点击事件这类异步操作，浏览器会把对应工作交给 Web APIs 或其他线程处理。等结果准备好，再把回调放入任务队列。

事件循环不断检查：调用栈是否为空？如果为空，就从任务队列里取出下一个任务执行。

这里还有一个关键细节：微任务。

Promise.then、queueMicrotask、MutationObserver 这类回调通常进入微任务队列。每执行完一个宏任务，浏览器会先清空微任务队列，再进入渲染或下一个任务。

所以常见顺序是：同步代码先执行，微任务随后执行，宏任务再执行。

这套机制的第一性原理是：

浏览器必须在单线程 JavaScript 语义、异步 I/O、用户输入和页面渲染之间建立调度秩序。

如果 JavaScript 长时间占用主线程，浏览器就无法及时处理输入和渲染，用户看到的就是页面卡顿。

第七部分：用户交互如何触发页面更新

当用户点击按钮、滚动页面、输入文字时，浏览器会把底层输入事件转换成 DOM 事件，分发给对应节点上的事件监听器。

事件分发通常包含捕获、目标、冒泡三个阶段。捕获阶段从根节点向目标节点传播，目标阶段到达实际触发节点，冒泡阶段再从目标节点向外传播。

JavaScript 事件处理函数可能修改 DOM 或 CSSOM。比如点击按钮后添加一个 class，或者插入一段列表。

一旦结构或样式变化，浏览器需要判断哪些渲染工作失效。

如果只是文字颜色变化，可能只需要重新绘制。

如果新增一个元素导致其他元素位置变化，就需要重新布局。

如果只是一个独立图层的 transform 变化，可能只需要重新合成。

现代浏览器会尽量做增量更新，而不是每次都重建整个页面。这就是脏标记、局部重排、分层合成等优化思想的意义。

从开发者角度，性能优化不是背技巧，而是理解渲染成本模型。

减少不必要的 DOM 规模，是减少结构计算成本。

减少复杂选择器和频繁样式变化，是减少样式计算成本。

避免同步读取和写入布局，是减少强制重排。

使用 transform 和 opacity 做动画，是尽量把工作留在合成阶段。

把长任务拆开，是避免主线程长时间阻塞输入和渲染。

第八部分：安全模型是浏览器存在的底线

如果浏览器只追求功能，它可以允许网页随便访问本地文件、读取任意网站 Cookie、扫描内网、控制摄像头。但那样浏览器就不可能成为通用平台。

浏览器安全模型的核心，是默认不信任网页。

同源策略是最基础的边界。所谓同源，是协议、域名、端口都相同。一个源的脚本，默认不能随意读取另一个源的敏感数据。

注意，同源策略不是禁止跨站请求。表单、图片、脚本标签都可能发起跨站请求。它真正限制的是跨源读取响应内容的能力。

CORS，也就是跨源资源共享，是服务器主动放开的机制。服务器通过响应头告诉浏览器：哪些源可以读取我的响应，哪些方法和请求头被允许。

Cookie 的 SameSite、HttpOnly、Secure 等属性，则是在身份凭证层面减少攻击面。HttpOnly 防止脚本读取 Cookie，Secure 要求 Cookie 只在 HTTPS 下发送，SameSite 用来缓解跨站请求伪造。

HTTPS 则解决传输链路上的身份、加密和完整性问题。没有 HTTPS，中间人可以窃听或篡改页面内容。对浏览器来说，安全页面的前提不是页面写得好，而是传输链路没有被劫持。

沙箱和站点隔离则进一步把不可信页面限制在更小的执行环境里。即使渲染进程被攻击，攻击者也不应该轻易突破到操作系统或其他站点数据。

所以浏览器安全不是某一个功能，而是一组叠加边界：

网络层用 HTTPS 保护传输。

身份层用 Cookie 策略保护凭证。

数据层用同源策略和 CORS 限制读取。

执行层用沙箱和进程隔离限制破坏范围。

第九部分：把整条链路串起来

现在我们重新回答最开始的问题：浏览器如何把 URL 变成页面？

第一步，浏览器解析 URL，判断协议、主机、路径和参数。

第二步，它检查缓存。如果缓存可用，直接复用；否则通过 DNS 找到服务器 IP。

第三步，它建立连接。TCP 负责可靠传输，TLS 负责身份校验、加密和完整性保护。

第四步，它发送 HTTP 请求，拿到服务器返回的字节流。

第五步，它把 HTML 解析成 DOM，把 CSS 解析成 CSSOM，并构建 Render Tree。

第六步，它进行 Layout，计算每个元素的位置和尺寸。

第七步，它进行 Paint，生成绘制指令。

第八步，它进行 Composite，把不同图层合成为最终画面并显示到屏幕。

第九步，JavaScript 在事件循环的调度下执行，响应用户输入、网络回调、定时器和各种异步任务。

第十步，每次 DOM、CSSOM 或视觉属性发生变化，浏览器都会根据影响范围重新走必要的渲染阶段。

这一整套机制背后的本质，是浏览器在同时满足四个目标：

正确性：页面要符合 HTML、CSS、JavaScript 的语义。

性能：页面要尽快加载、尽快渲染、交互不能卡顿。

安全：远程代码不能随意破坏用户系统和其他网站数据。

兼容性：互联网上海量页面都应该尽可能正常运行。

浏览器的复杂性，正是来自这四个目标之间的权衡。

结尾

如果只背“输入 URL 后发生了什么”，你会记住很多零散术语：DNS、TCP、TLS、HTTP、DOM、CSSOM、Layout、Paint、事件循环。

但如果从第一性原理看，你会发现它们都在回答同一个问题：

如何把不可信的远程字节，安全、高效、正确地变成本机上的可交互应用？

DNS 解决“资源在哪里”。

TCP 和 TLS 解决“如何可靠、安全地取回来”。

HTTP 解决“客户端和服务器如何表达资源请求与响应”。

DOM 和 CSSOM 解决“如何理解结构和样式”。

Layout、Paint、Composite 解决“如何把页面画出来”。

JavaScript 和事件循环解决“如何让页面持续运行和响应交互”。

同源策略、CORS、HTTPS、沙箱和站点隔离解决“如何限制不可信代码的边界”。

所以浏览器不是一个简单软件，而是一个把网络、操作系统、编译器、图形学和安全模型压缩在一起的平台。

理解这一点，你再看前端性能优化、跨域、安全策略、事件循环、渲染阻塞，就不会再是一堆孤立的八股文，而是一套可以从底层推导出来的系统。

这就是浏览器真正的底层原理。